

CSSE3113 – Introduction to Software Engineering

Requirements Modelling

Requirement Analysis

- Requirements analysis
 - Specifies software's operational characteristics
 - Indicates software's interface with other system elements
 - Establishes constraints that software must meet
- Requirements analysis allows the software engineer (called an *analyst* or *modeler* in this role) to:
 - Elaborate on basic requirements established during earlier requirement engineering tasks
 - Build models that depict user scenarios, functional activities, problem classes and their relationships, system and class behavior, and the flow of data as it is transformed.

Rule of Thumb

- The model should focus on requirements that are visible within the problem or business domain. The level of abstraction should be relatively high.
- Each element of the analysis model should add to an overall understanding of software requirements and provide insight into the information domain, function and behavior of the system.
- Delay consideration of infrastructure and other non-functional models until design.
- Minimize coupling throughout the system.
- Be certain that the analysis model provides value to all stakeholders.
- Keep the model as simple as it can be.

Domain Analysis

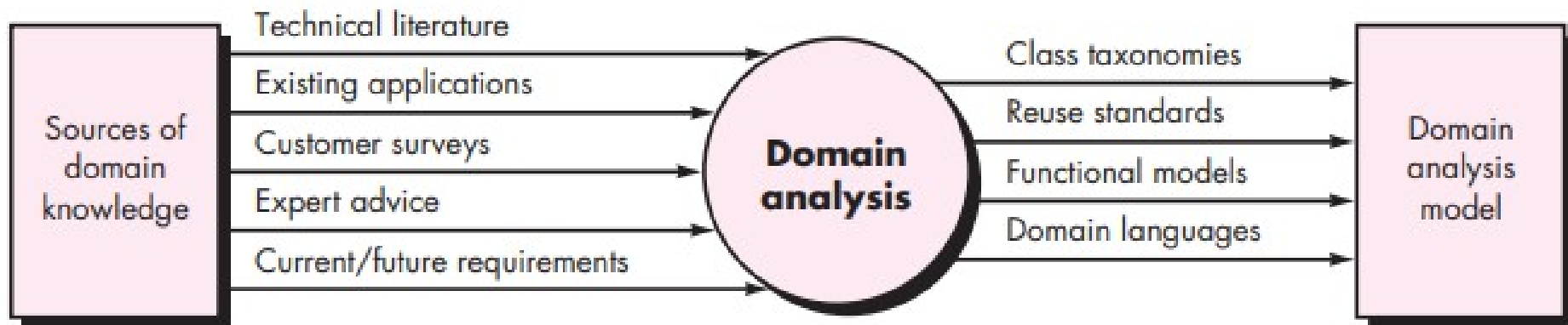
Software domain analysis is the identification, analysis, and specification of common requirements from a specific application domain, typically for reuse on multiple projects within that application domain . . .

[Object-oriented domain analysis is] the identification, analysis, and specification of common, reusable capabilities within a specific application domain, in terms of common objects, classes, subassemblies, and frameworks . . .

- ***Donald Firesmith***

Domain Analysis

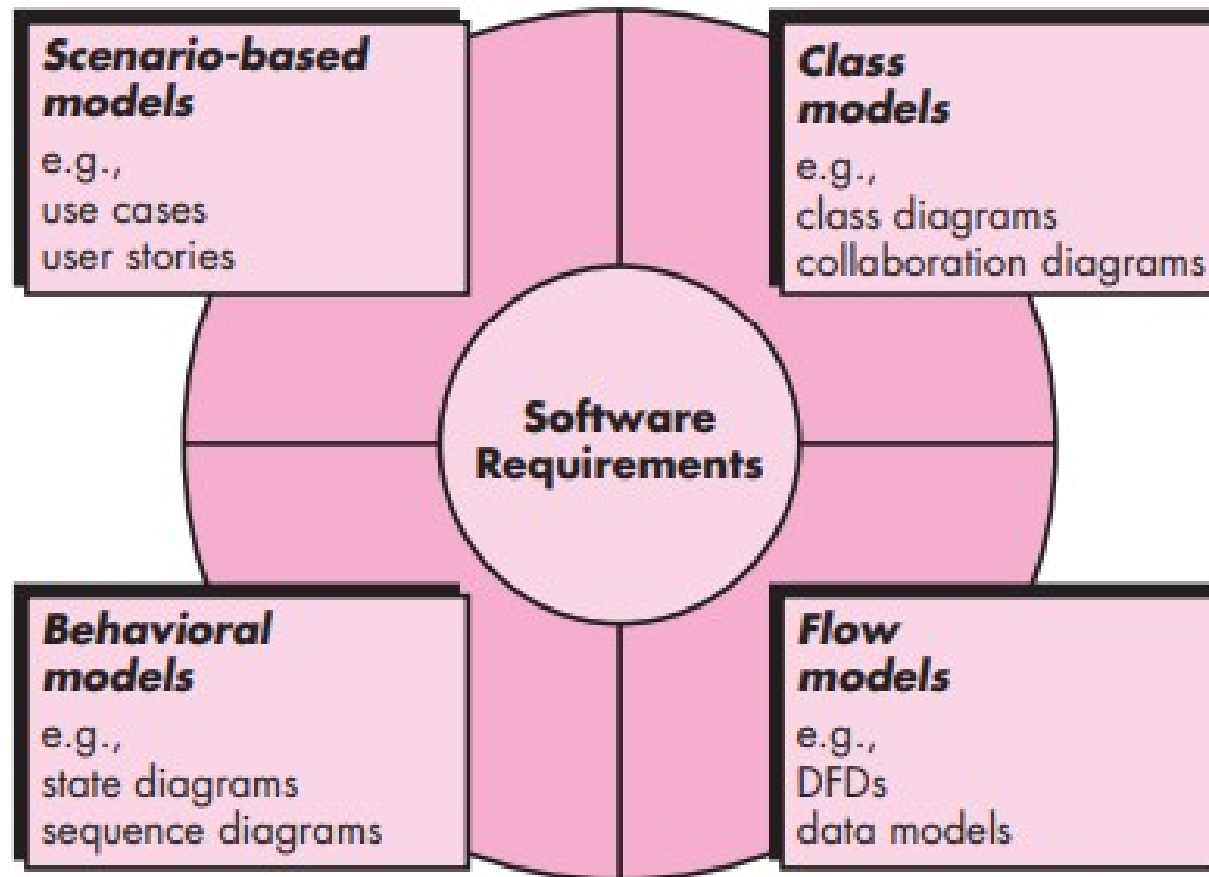
- Define the domain to be investigated.
- Collect a representative sample of applications in the domain.
- Analyze each application in the sample.
- Develop an analysis model for the objects.



Requirement Modelling

- The intent of the analysis/requirement model is to provide a description of the required informational, functional, and behavioral domains for a computer-based system.
- The model changes dynamically as you learn more about the system to build.
- Other stakeholders understand more about what they really require.
- There are different modes of representation that force you to consider requirements from different viewpoints---an approach that has a higher probability of uncovering omissions, inconsistencies, and ambiguity.

Requirement Modelling



Scenario-based Modelling

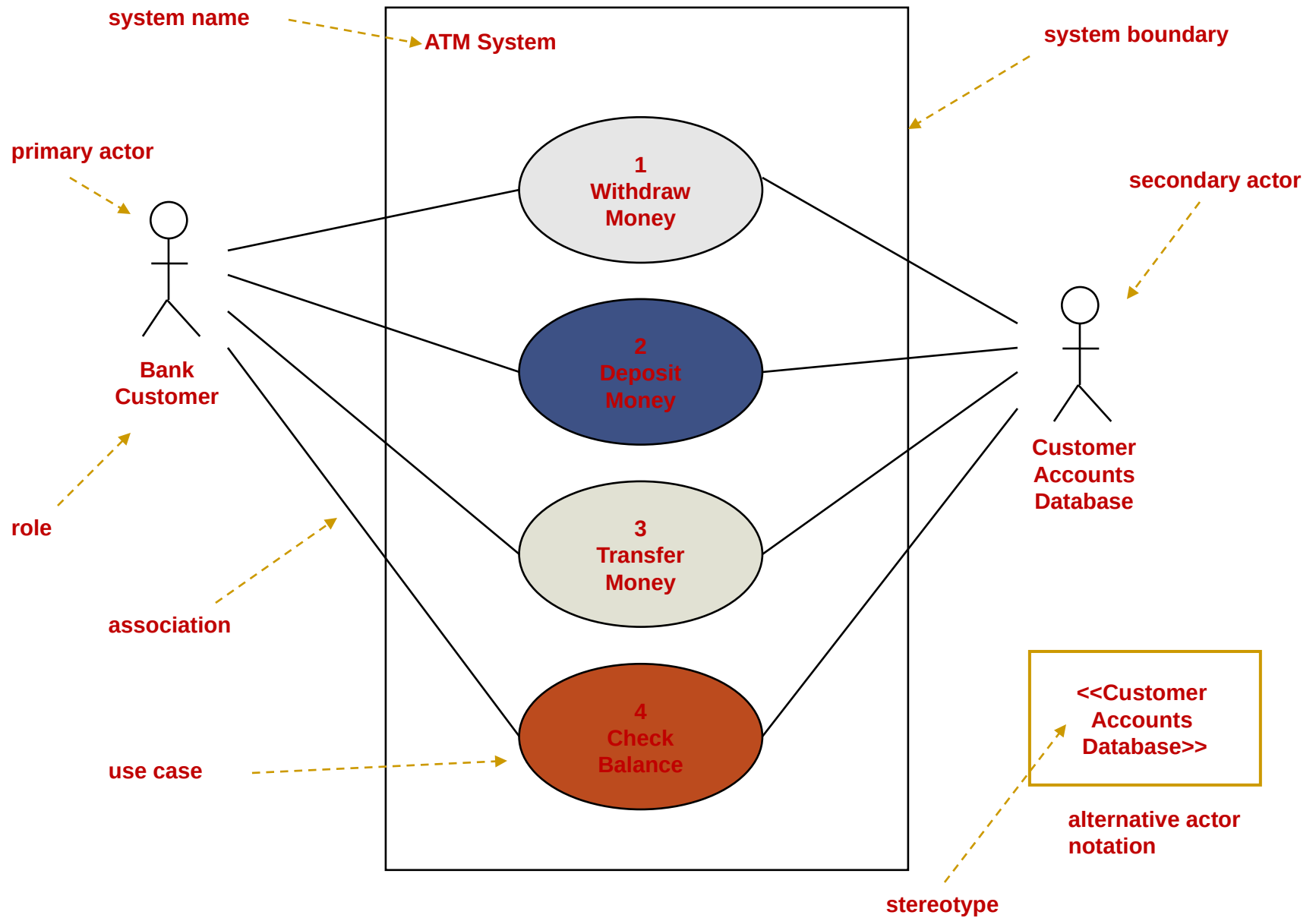
- The scenarios often called use cases, provide a description of how the system will be used.
- In essence, a use case tells a stylized story about how an end user interacts with the system under a specific set of circumstances.
- The story can be a narrative text, an outline of tasks or interactions, a template-based description, or a diagrammatic representation.

Scenario-based Modelling- Use case

- A collection of user scenarios that describe the thread of usage of a system
- Each scenario is described from the point-of-view of an “actor”—a person or device that interacts with the software in some way (not necessarily users) to External entities.
- For example, a machine in four different modes of programming, test, monitoring and troubleshooting modes. Four actors can be defined. Programmer, tester, monitor, and troubleshooter.
- In some cases, the machine operator can play all of these roles. Or different people can play different roles.

Scenario-based Modelling- Use case

- Once actors are defined, use cases can be developed. Each scenario answers the following questions:
 - Who is the primary actor, the secondary actor (s) (to support the primary actors)?
 - What are the actor's goals?
 - What preconditions should exist before the story begins?
 - What main tasks or functions are performed by the actor?
 - What extensions might be considered as the story is described?
 - What variations in the actor's interaction are possible?
 - What system information will the actor acquire, produce, or change?
 - Will the actor have to inform the system about changes in the external environment?
 - What information does the actor desire from the system?
 - Does the actor wish to be informed about unexpected changes?



Class-based Modelling

- It shows the classes of the system, their inter-relationships, and the operations and attributes of the classes
- Explore domain concepts in the form of a domain model
- Analyze requirements in the form of a conceptual/analysis model

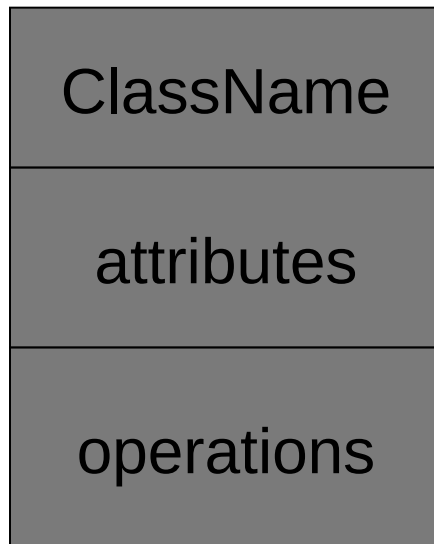
Class-based Modelling

- Class-based modeling represents:
 - **Objects** that the system will manipulate
 - **Operations** (also called methods or services) that will be applied to the objects to effect the manipulation
 - **Relationships** (some hierarchical) between the objects
 - **Collaborations** that occur between the classes that are defined.
- The elements of a class-based model include classes and objects, attributes, operations, and collaboration diagrams.

Identifying Classes

- Examining the usage scenarios developed as part of the requirements model and perform a "grammatical parse".
 - Classes are determined by underlining each noun or noun phrase and entering it into a simple table.
 - Synonyms should be noted.
 - If the class (noun) is required to implement a solution, then it is part of the solution space; otherwise, if a class is necessary only to describe a solution, it is part of the problem space.
- But what should we look for once all of the nouns have been isolated?

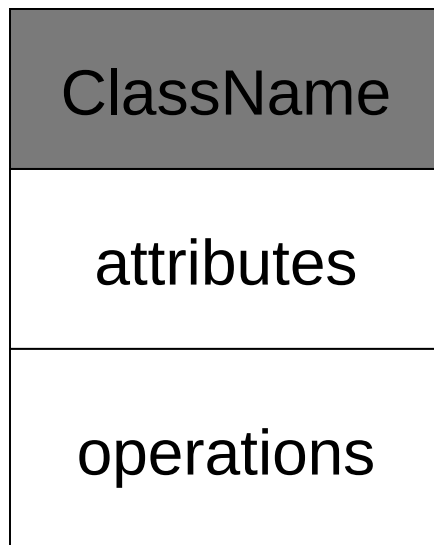
Class



A *class* is a description of a set of objects that share the same attributes, operations, relationships, and semantics.

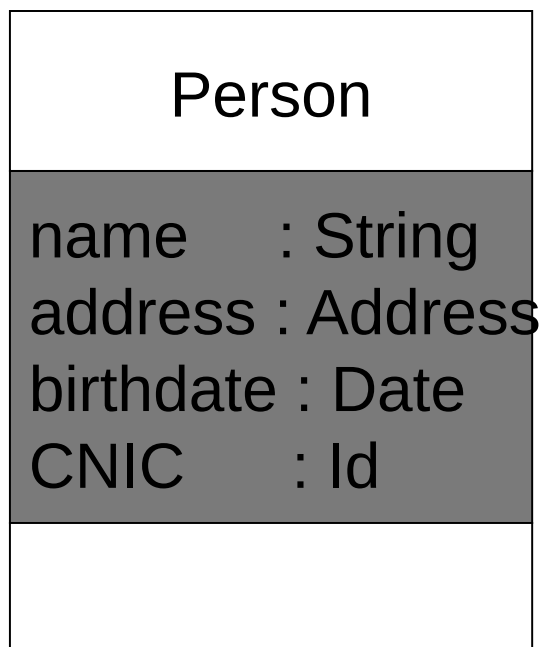
Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations in separate, designated compartments.

Class Name



The name of the class is the only required tag in the graphical representation of a class. It always appears in the top-most compartment.

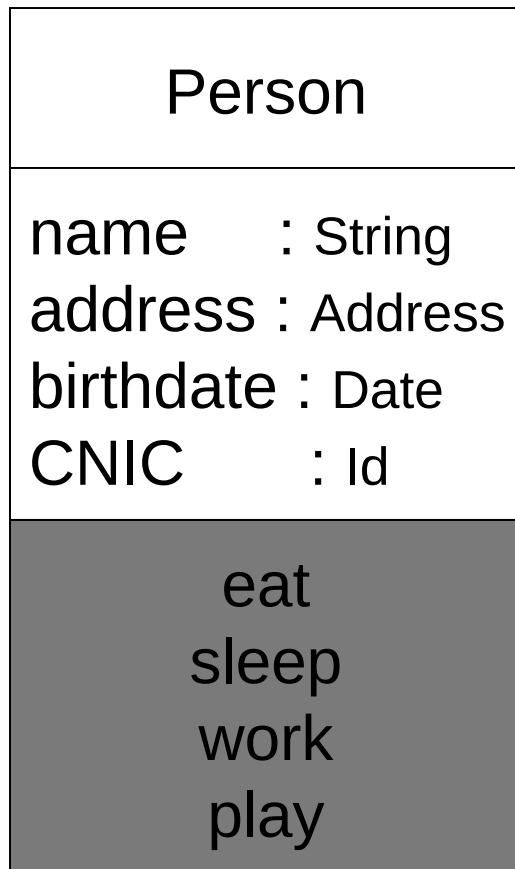
Class Attributes



An *attribute* is a named property of a class that describes the object being modeled.

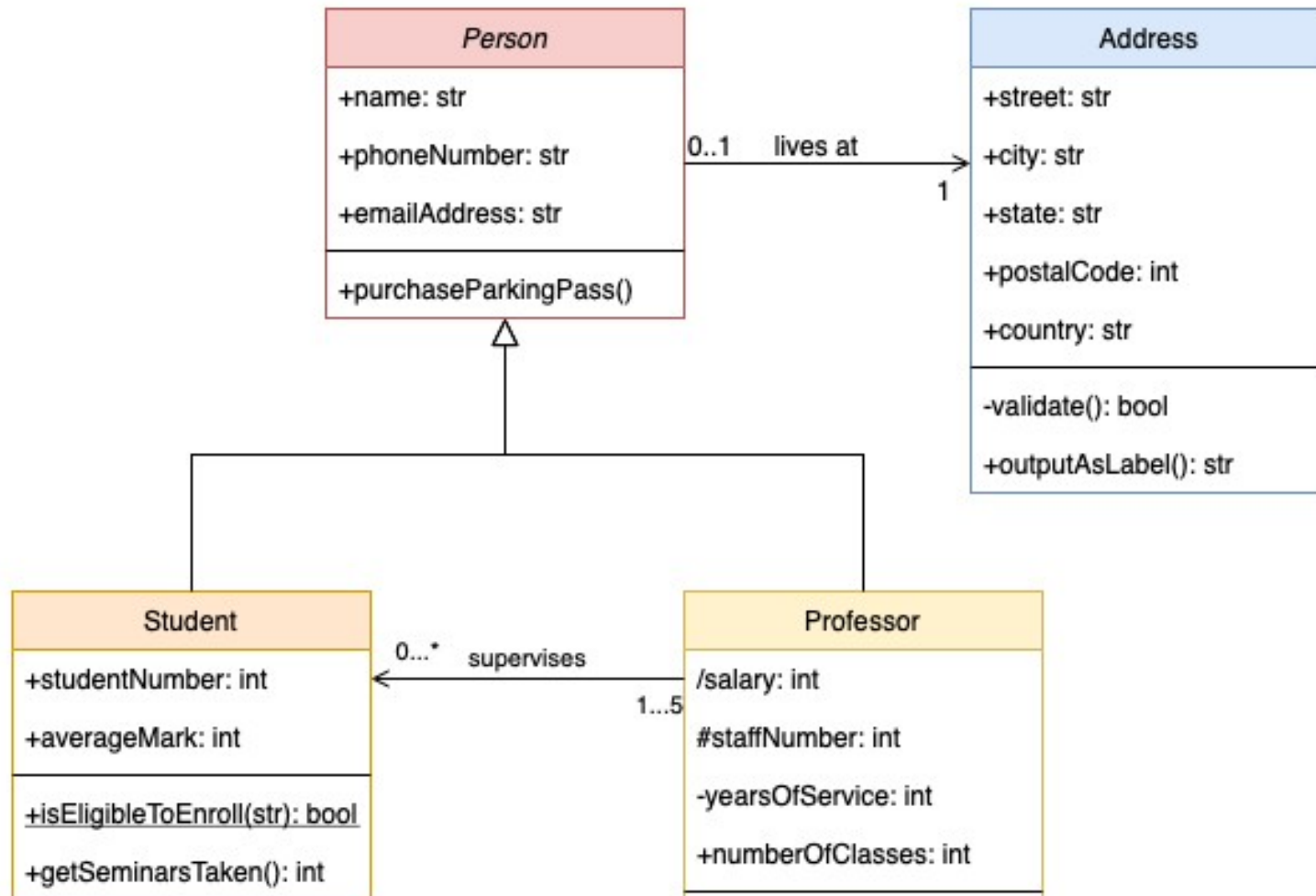
In the class diagram, attributes appear in the second compartment just below the name-compartment.

Class Operations



Operations describe the class behavior and appear in the third compartment.

Class Diagram - Example

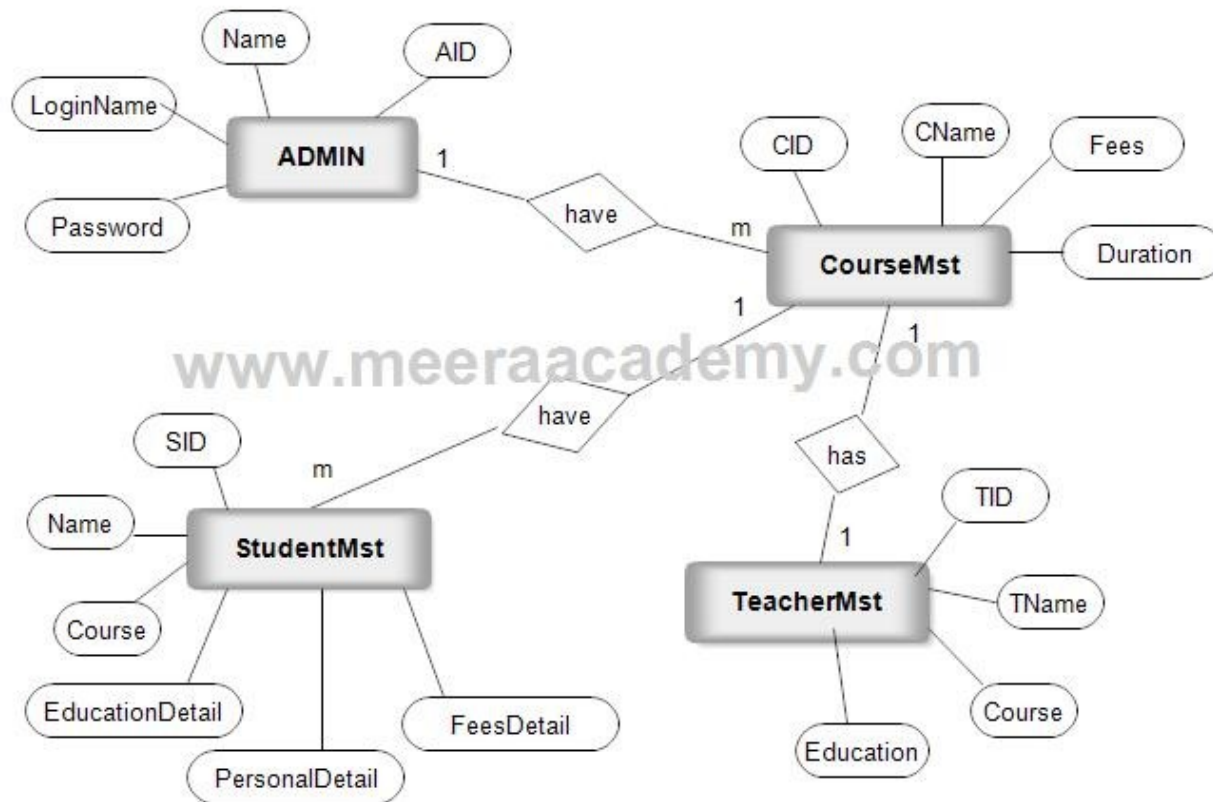


ER to class diagram

- Here's a breakdown of the process:
- **Entities to Classes:** Each entity in the ERD becomes a class in the class diagram.
- **Attributes to Class Attributes:** Attributes of the entities become attributes of the corresponding classes.
- **Relationships to Associations:** Relationships between entities are represented as associations between the corresponding classes.
- **Cardinality:** Consider the cardinality of relationships (e.g., one-to-one, one-to-many, many-to-many) and represent them in the class diagram.
- **Additional UML Elements:**
 - **Inheritance:** If you have inheritance relationships in your domain, represent them in the class diagram using inheritance lines.
 - **Methods:** Add methods (operations) to the classes that are relevant to the application domain.
 - **Private Attributes:** Consider which attributes should be private and represent them accordingly in the class diagram.

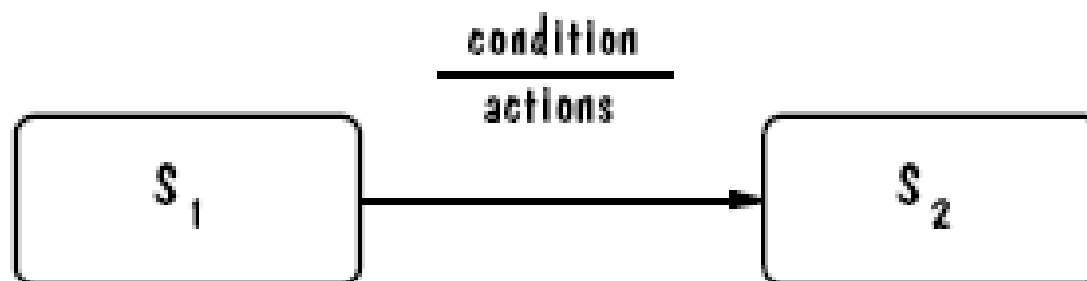
ERD to class diagram

ER Diagram Student Management System



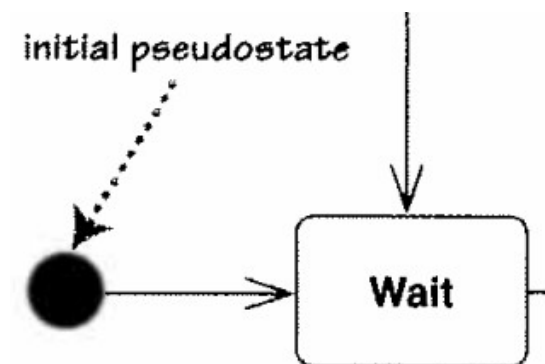
Behavioral Modelling

- **State Diagram** is a type of behavioral model
- Depicts data and behavior of a single object throughout its lifetime.
- **Elements of State Diagram**
 - Set of states (including an initial start state)
 - Transitions between states
 - Entire diagram is drawn from that object's perspective

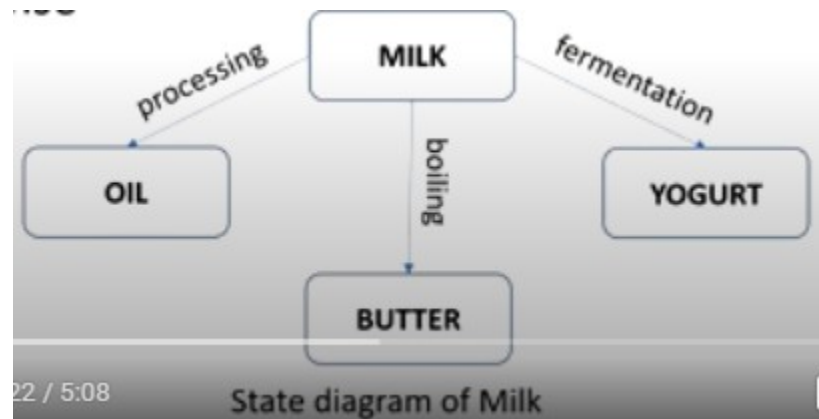


States

- **State:** Conceptual description of the data in the object
 - Represented by object's field values
- Entire diagram is drawn from the central object's perspective
 - Only include states / concepts that this object can see and influence
 - Don't include every possible value for the fields; only ones that are conceptually different

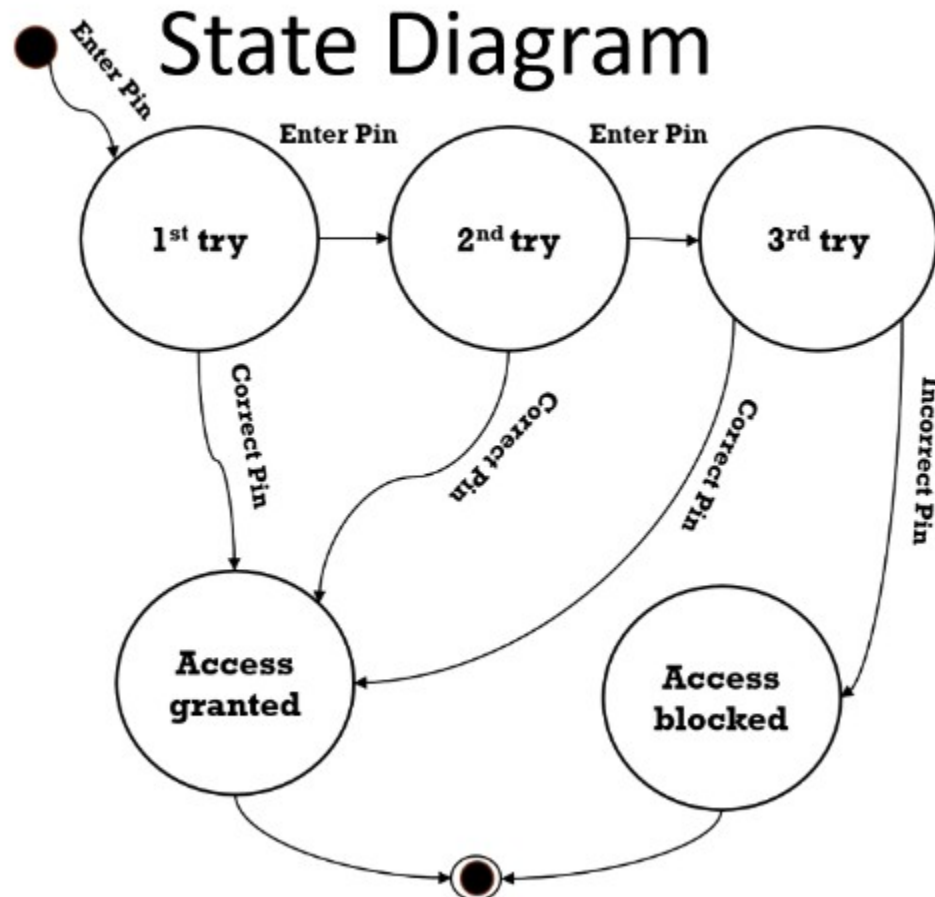


Example



- Let's consider an ATM system function where if the user enters the invalid password three times the account will be locked.
- In this system, if the user enters a valid password in any of the first three attempts the user will be logged in successfully. If the user enters the invalid password in the first or second try, the user will be asked to re-enter the password. And finally, if the user enters incorrect password 3rd time, the account will be blocked.

Example



Class activity

Design a state diagram for a cell phone call. States might include "Idle," "Ringing," "In Call," and "Call Ended." Define transitions triggered by events such as receiving an incoming call, answering a call, and ending a call.

Example Scenario

- I want to keep my valuables in safe that's hard to find. So to reveal the lock to the safe, I have to remove a strategic candle from its holder, but this will reveal the lock only when the door is closed.
- Once I can see the lock I can insert my key to open the safe. For extra safety, I make sure that I can open the safe only I replace the candle first. If a thief neglects this precaution, I 'll unleash the nasty monster to devour him.

State Diagram

